

Automated Software Testing Part 2

Automated Reduction of Bug-Triggering SQL Queries

Edouard Mignan (21806807)
Eduard Mihai (25933516)

June 11, 2026

1 Technical Description

In this section, we will describe the methods implemented in our reducer and the limitations of our approach.

Our reducer is an implementation of Hierarchical Delta Debugging [1] (particularly, a variant called HDD*). Contrary to Delta Debugging [5], which handles input as a raw array of data to perform reduction, Hierarchical Delta Debugging (HDD) applies to structured inputs and uses knowledge of that structure to perform better reductions with fewer generated test cases. Our reducer works by parsing a SQL query into a parse tree and applying Delta Debugging to each level of that tree, from the highest to the lowest, in order to prune irrelevant nodes, and finally reconstruct the reduced query.

We implemented HDD* (HDD with multiple passes until no more reduction is possible) with some termination criteria to improve performance. In particular, our reducer will compare changes between the beginning and the end of a pass and stop prematurely if the improvement is inferior to 1%. It also performs a fixed maximum number of passes and times out if it takes too long to finish. In all cases, the program will finish with the best reduced query found so far.

We tested multiple tools to parse SQL for our reducer, and after some benchmarks, settled on ANTLR4 [2], as the implementation using it had the best performance and reduction quality on average. Other tools (tree-sitter, sqlglot) either suffered from inferior performance or parsing errors on a large number of benchmark queries, making reductions using HDD impossible.

The approach we took has limitations. Since HDD works simply by keeping or removing subtrees or nodes, it often produces and tests syntactically invalid queries. This leads to lower performance and worse reductions compared to more advanced, syntactically-aware approaches such as Perses [3]. This approach is also limited in terms of parallelisability to improve performance since each iteration of Delta Debugging relies on the previous iteration’s result. Other approaches, such as Probabilistic Delta Debugging [4], do not suffer from this limitation, which is especially important when large inputs are concerned.

2 Evaluation

After running our reducer with the provided test queries and oracles, we obtained the following results from Table 1.

Query	Original Size (tokens)	Reduced Size (tokens)	Token Reduction (%)	Time Taken (seconds)
1	216	180	16.67	23.37
2	481	94	80.46	4.25
3	1564	43	97.25	2.43
4	955	115	87.96	10.73
5	68	43	36.76	2.86
6	3810	104	97.27	15.67
7	619	29	95.32	0.71
8	1694	170	89.96	9.98
9	1245	36	97.11	1.18
10	259	48	81.47	1.80
11	114	81	28.95	3.96
12	3818	80	97.90	6.90
13	287	81	71.78	6.24
14	10787	870	91.93	60.30
15	224	105	53.12	5.76
16	5051	138	97.27	26.13
17	10559	291	97.24	0.74
18	209	137	34.45	18.34
19	211	81	61.61	3.25
20	9539	331	96.53	60.23

Table 1: Reducer Benchmark Results

References

- [1] Ghassan Misherghi and Zhendong Su. HDD: Hierarchical delta debugging. In *Proceedings of the 28th International Conference on Software Engineering, ICSE '06*, pages 142–151, 2006.
- [2] Terence Parr. ANTLR (another tool for language recognition), version 4. <https://www.antlr.org>, 2013. Accessed: 2026-06-11.
- [3] Chengnian Sun, Yuanbo Li, Qirun Zhang, Tianxiao Gu, and Zhendong Su. Perses: syntax-guided program reduction. In *Proceedings of the 40th International Conference on Software Engineering, ICSE '18*, page 361–371, New York, NY, USA, 2018. Association for Computing Machinery.
- [4] Guancheng Wang, Ruobing Shen, Junjie Chen, Yingfei Xiong, and Lu Zhang. Probabilistic delta debugging. In *Proceedings of the 29th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering, ESEC/FSE 2021*, page 881–892, New York, NY, USA, 2021. Association for Computing Machinery.
- [5] Andreas Zeller and Ralf Hildebrandt. Simplifying and isolating failure-inducing input. *IEEE Transactions on Software Engineering*, 28(2):183–200, 2002.